FH JOANNEUM - University of Applied Sciences

Pods Are Not Shards: An Empirical Comparison of Kubernetes-Native and Topology-Aware Operator-Based Autoscaling for Stateful Redis Clusters

Bachelorarbeit

zur Erlangung des akademischen Grades

„Bachelor of Science in Engineering“

eingereicht am Studiengang Wirtschaftsinformatik

Studienrichtung: Wirtschaftsinformatik

Verfasst von: Johann Krischan Hipolito

Betreuung: Michael Nestler

Graz, 2026

FH | JOANNEUM
University of Applied Sciences

Table of Contents

List of Figures

List of Tables

List of Abbreviations

API Application Programming Interface

YAML YAML Ain’t Markup Language

KEDA Kubernetes Event-Driven Autoscaling

CRD Custom Resource Definition

HTTP Hypertext Transfer Protocol

HPA Horizontal Pod Autoscalar

JS JavaScript

List of Symbols

Hier steht das Symbolverzeichnis.

List of Formulas

Hier steht das Formelverzeichnis.

Kurzfassung

Hier steht die Kurzfassung auf Deutsch.

Abstract

Here is the abstract in English.

1 Introduction

1.1 Cloud Computing and the Rise of Container Orchestration

The rapid evolution of modern software systems has fundamentally transformed the way infrastructure is designed, deployed, and operated. At the heart of this transformation lies *cloud computing* — a paradigm that enables on-demand access to a shared pool of configurable computing resources, including networks, servers, storage, applications, and services, that can be rapidly provisioned and released with minimal management effort (Mell and Grance, 2011). This model has shifted the industry away from static, on-premises data centres toward elastic, scalable environments that can react dynamically to workload demands.

As organisations increasingly adopt cloud-native architectures, the notion of designing systems that are resilient, scalable, and operationally excellent has become central to engineering practice. The AWS Well-Architected Framework, for instance, codifies these concerns into a set of established design principles for container-based workloads, emphasising repeatability, automation, and observability as first-class concerns in modern infrastructure (Amazon Web Services, 2022).

A critical enabler of cloud-native architectures is the *container* — a lightweight, self-contained unit of software that packages application code together with its dependencies, configuration, and runtime environment. Unlike traditional virtual machines, containers share the host operating system kernel and therefore incur significantly less overhead while still providing process-level isolation (Docker Inc., 2016). This characteristic makes containers especially well suited to microservice architectures, where large monolithic applications are decomposed into smaller, independently deployable services that can be scaled individually based on demand.

1.2 Kubernetes as the Standard for Container Orchestration

Managing containerised workloads at scale introduces significant operational complexity. Scheduling containers across a fleet of machines, maintaining desired application state, performing rolling updates, and recovering from failures all require robust automation. It is in response to exactly these challenges that *Kubernetes* — also referred to as K8s — emerged as the dominant container orchestration platform.

Originally designed and used internally at Google as the *Borg* system, Kubernetes was open-sourced in 2014 and donated to the Cloud Native Computing Foundation (CNCF), where it has since grown into the second-largest open-source project in the world (IBM, 2024). Kubernetes provides a declarative Application Programming Interface (API) for defining the desired state of an application cluster; its control plane continuously reconciles the observed state of the system against this desired state, automatically scheduling pods, managing service discovery, and orchestrating storage (Kubernetes Documentation, 2024c).

To reduce operational complexity further, lightweight Kubernetes distributions such

as *K3s* — developed by Rancher and certified by the CNCF — have gained widespread adoption. K3s packages the full Kubernetes feature set into a single binary with a significantly reduced memory footprint, making it suitable for edge computing, IoT environments, and resource-constrained infrastructure (K3s Project, 2024). In the context of this thesis, K3s serves as the underlying cluster management layer for the experimental infrastructure under investigation.

1.3 The Challenge of Scaling Stateful Applications

While Kubernetes excels at managing stateless workloads — where any replica of a service can handle any request independently — scaling *stateful applications* presents a distinct set of challenges. Stateful applications, such as in-memory databases and message queues, maintain persistent data or internal state that must be carefully managed across the lifecycle of individual pods. Kubernetes provides the `StatefulSet` primitive to address these requirements, offering stable network identities, ordered deployment and scaling, and persistent volume binding (Kubernetes Documentation, 2024e).

Redis is a canonical example of such a stateful application. It is a high-performance, in-memory data store widely used for caching, session management, and real-time analytics (Redis Ltd., 2024a). When deployed as a Redis Cluster across multiple nodes, it uses sharding to distribute data and replication to ensure fault tolerance. Scaling a Redis Cluster in a Kubernetes environment is non-trivial: adding or removing nodes requires coordinated resharding of data slots, careful management of cluster topology, and maintenance of quorum — operations that introduce significant risk if performed reactively under resource pressure.

Recent work from the CNCF community has highlighted these concerns, noting that while Kubernetes operators can partially automate lifecycle management for Redis, the path toward robust, cloud-native stateful services remains an open challenge (Cloud Native Computing Foundation, 2024).

1.4 Autoscaling Strategies: From Reactive to Predictive

Kubernetes provides native support for horizontal scaling through the *Horizontal Pod Autoscaler* (HPA), which monitors CPU and memory utilisation metrics and adjusts the number of pod replicas accordingly (Kubernetes Documentation, 2024a). While the HPA is effective for stateless services under gradually increasing load, its reactive nature introduces an inherent latency: scaling decisions are made only *after* resource thresholds have already been breached. For latency-sensitive or stateful workloads, this lag can result in degraded performance, pod restarts, or cascading failures during traffic spikes (Thoras, 2024).

An increasingly popular alternative is *Kubernetes Event-Driven Autoscaling* (Kubernetes Event-Driven Autoscaling (KEDA)), a CNCF-graduated project that extends the standard HPA to support event- and metric-driven scaling from a rich ecosystem of external sources, including message queues, databases, and custom Prometheus metrics (KEDA

Project, 2024a; Cloud Native Computing Foundation, 2023). By decoupling scaling decisions from simple resource utilisation and enabling them to respond to application-level events, KEDA enables a more proactive and context-aware approach to scaling. This is particularly relevant for stateful workloads where pre-emptive resource provisioning may prevent the cluster-level disruptions that reactive scaling cannot avoid.

1.5 Research Objective and Thesis Structure

This thesis investigates the performance differences between two autoscaling strategies for stateful applications deployed in a Kubernetes cluster managed by K3s. The first strategy employs the native Horizontal Pod Autoscaler in a *reactive* configuration, scaling a Redis Cluster in response to CPU and memory resource expenditure. The second strategy employs a *custom KEDA-based operator* in a predictive, event-driven configuration, triggering scaling decisions based on application-level metrics sourced from a Prometheus observability stack.

The experimental infrastructure consists of a FastAPI-based entrypoint service (FastAPI, 2024b), a Redis Cluster with a minimum of three nodes, an observability stack comprising Prometheus and Grafana deployed via Helm, and two scaling configuration manifests representing the naive HPA and KEDA-driven approaches respectively. All components are deployed on a K3s cluster, providing a reproducible and resource-constrained environment representative of real-world edge and on-premises deployments.

The remainder of this thesis is structured as follows. Chapter 2 details the differences between the automatic provisioning of stateless and stateful applications, a well as the issues that arise when scaling stateful workloads. Chapter 4 provides the theoretical background on Kubernetes architecture, autoscaling mechanisms, stateful workloads, and the Redis Cluster model. Chapter **??** describes the system design and experimental setup. Chapter 7 presents the experimental results and performance analysis. Chapter 8 summarises the findings and discusses directions for future work.

2 Scaling Stateful and Stateless Applications in Kubernetes

A fundamental distinction in distributed systems design is that between *stateless* and *stateful* applications, and this distinction has direct consequences for how workloads are deployed, managed, and scaled inside a Kubernetes cluster. Understanding this divide is essential for motivating the core research question of this thesis.

2.1 Stateless Applications and Horizontal Scalability

A stateless application does not retain any client-specific data between requests. Each request is self-contained and can be served by any replica of the application without prior knowledge of previous interactions (Red Hat, 2025). This architectural property makes stateless services trivially scalable: because every replica is functionally identical, the Kubernetes scheduler can start or terminate any number of them at any time without coordination. Kubernetes `Deployments` and `ReplicaSets` are the canonical workload primitives for stateless applications, providing identical, interchangeable pod replicas with no notion of stable identity or ordered lifecycle (Kubernetes Documentation, 2024e).

Horizontal scaling of a stateless service via the Horizontal Pod Autoscaler (HPA) is therefore straightforward: when aggregate CPU or memory utilisation exceeds a configured threshold, the HPA controller issues a scale-out instruction to the relevant `Deployment`, and the new replicas are scheduled and become available without any data migration, topology reconfiguration, or disruption to existing replicas (Kubernetes Documentation, 2024a). Conversely, when load decreases, replicas can be terminated immediately since no in-replica state is lost. This symmetry between scale-out and scale-in is what makes the reactive HPA model well-suited to stateless workloads.

2.2 The Challenges of Scaling Stateful Applications

Stateful applications, by contrast, maintain persistent data or internal state that is specific to individual instances and must survive pod restarts, rescheduling events, and scaling operations. This category includes relational databases, key-value stores, message queues, and consensus systems. Kubernetes provides the `StatefulSet` workload primitive to accommodate these requirements. Unlike a `Deployment`, a `StatefulSet` guarantees that each pod receives a stable, unique network identity (e.g., `redis-0`, `redis-1`), a dedicated `PersistentVolumeClaim` that is bound across restarts, and an ordered, sequential lifecycle for pod creation and termination (Kubernetes Documentation, 2024e, Portworx, 2024).

These guarantees, while necessary for correctness, fundamentally complicate autoscaling. The following challenges are particularly relevant in the context of this thesis:

Ordered deployment and termination. `StatefulSet` pods are created and deleted in strict ordinal order. When scaling out from n to $n + 1$ replicas, Kubernetes will not proceed until the n-th pod is fully *Ready*. This serialisation ensures application-level invariants (such as cluster quorum) are maintained but introduces latency that can be problematic under sudden load spikes (Kubernetes Documentation, 2024e). A reactive autoscaler such as the HPA, which fires only after a resource threshold has already been breached, is therefore poorly positioned to absorb burst traffic for stateful workloads: by the time new replicas are ordered, scheduled, initialised, and joined to the cluster, the performance degradation may already be severe.

Persistent volume provisioning. Each new pod added to a `StatefulSet` requires a new `PersistentVolumeClaim` to be dynamically provisioned by the storage backend. Depending on the underlying storage class and infrastructure, this provisioning step can itself introduce non-trivial latency, further extending the time-to-ready for new stateful replicas compared to stateless counterparts (Portworx, 2024).

Cluster topology and data rebalancing. For distributed databases and in-memory stores that shard data across their nodes — such as Redis Cluster — adding or removing a node is not merely a matter of starting or stopping a process. Redis Cluster partitions its keyspace across exactly $16\,384$ hash slots, each of which is assigned to a primary node (Redis Ltd., 2024c). When a new node is added, the cluster operator must *reshard* a subset of hash slots from existing nodes to the new one; when a node is removed, its slots must first be migrated away. This resharding process involves live data movement between nodes and, if performed reactively under resource pressure, carries a non-trivial risk of increased latency, partial unavailability, or data inconsistency (Google Cloud, 2023).

Quorum and availability constraints. Many stateful distributed systems depend on a quorum of nodes being available to elect a leader, acknowledge writes, or service reads. Redis Cluster requires at least one primary per shard to be available, and Kubernetes `PodDisruptionBudget` (PDB) resources are the mechanism used to express minimum availability constraints during voluntary disruptions such as autoscaling events (Kubernetes Documentation, 2024b). Aggressive scale-in triggered by a reactive autoscaler can violate quorum constraints if PDBs are not carefully configured, risking full cluster unavailability.

The HPA and stateful workloads. While the HPA technically supports `StatefulSets` as scaling targets, its reactive, threshold-based model is widely considered inappropriate for database and cache workloads (Vazquez, 2026). The HPA operates on *lagging indicators*: it reacts to resource pressure that has already materialised. For a stateful application such as Redis, where high memory pressure can degrade cache hit rates and high CPU pressure can stall replication, by the time the HPA observes a breach and a new node has completed resharding, a significant window of degraded performance will have already elapsed. Furthermore, the HPA's default synchronisation period of

15 seconds (KEDA Project, 2024d) means that multiple polling cycles may pass before a scaling decision is acted upon, compounding the lag.

These challenges collectively motivate the investigation of a more *proactive*, event-driven autoscaling strategy for stateful workloads, as explored in this thesis through the use of KEDA.

3 Hypothesis: The Operator Pattern with KEDA as a Superior Autoscaling Strategy for Stateful Workloads

The limitations of the reactive HPA model described in Section 2.2 motivate the central hypothesis of this thesis: that a *proactive, event-driven autoscaling strategy*, implemented through a custom Kubernetes operator in combination with KEDA, is a superior method of scaling stateful workloads compared to the native HPA baseline.

3.1 The Operator Pattern as a Foundation for Stateful Automation.

The Kubernetes *operator pattern* was conceived precisely to address the gap between the generic automation that Kubernetes provides out of the box and the deep, application-specific knowledge required to manage complex stateful systems reliably. According to the official Kubernetes documentation, the operator pattern "aims to capture the key aim of a human operator who is managing a service or set of services", encoding that operational expertise into a software controller that participates in the standard Kubernetes reconciliation loop (Kubernetes Documentation, 2024d). Operators extend the Kubernetes API through *Custom Resource Definitions* (CRDs) and an accompanying controller that continuously reconciles the observed state of the custom resource against its declared desired state.

Critically, the CNCF Operator White Paper explicitly acknowledges that "Kubernetes primitives were not built to manage state by default" and that "relying on Kubernetes primitives alone brings difficulty managing stateful application" lifecycle (CNCF TAG App Delivery, 2023). The operator pattern is therefore not merely a convenience — it is the architecturally recommended approach for encoding domain-specific concerns such as data resharding, replication topology maintenance, and quorum-safe scaling into a Kubernetes-native automation layer. For a Redis Cluster deployment, a custom operator is capable of expressing and enforcing exactly the application-level invariants that a generic HPA controller cannot: for example, ensuring that a scale-out event is only issued once the cluster is fully healthy, and that the requisite resharding of hash slots is coordinated atomically with the scheduling of new `StatefulSet` pods.

3.2 KEDA's Event-Driven Model as a Proactive Trigger.

Complementing the operator pattern, KEDA introduces an event-driven trigger layer that decouples scaling decisions from raw infrastructure metrics. By consuming application-level signals exposed via Prometheus — such as request rate, cache hit ratio, or queue depth — KEDA's `ScaledObject` can issue a scaling recommendation *before* CPU or memory thresholds are breached, effectively shifting the scaling decision upstream in the load curve (KEDA Project, 2024d). This is in direct contrast to the HPA model, in which scaling decisions are always downstream of resource exhaustion. The additional configurability of KEDA's `ScaledObject` — including fine-grained control over polling intervals, cooldown periods, and minimum replica counts (Medium /

emreblblvv, 2024) — further allows the autoscaling behaviour to be tuned to the specific performance characteristics of a Redis Cluster, where aggressive scale-in during low-traffic periods is inadvisable due to the cost of subsequent resharding.

3.3 Supporting Evidence from the Literature.

The hypothesis is supported by a growing body of academic and industry research. Mondal et al. identified that the HPA's reliance on CPU and memory as lagging indicators makes it "incapable of foreseeing upcoming workload spikes", leading to Quality of Service violations, long-tail latency, and wasted resources (Wanigasooriya and Ekanayake, 2026). Wanigasooriya and Ekanayake similarly demonstrated in the NimbusGuard study that proactive autoscaling frameworks "translate into superior performance and cost efficiency compared to existing reactive methods" when evaluated alongside standard HPA and KEDA baselines under identical, phased load patterns (Wanigasooriya and Ekanayake, 2026). Although that study positions KEDA itself as part of the reactive baseline relative to a deep-learning agent, it nevertheless confirms that event-driven, application-aware autoscaling consistently outperforms pure resource-metric scaling — a finding that directly supports the use of KEDA-sourced Prometheus metrics as the trigger mechanism in the present thesis.

Furthermore, Toka et al. identified three structural drawbacks in the current Kubernetes reactive scaling process: its purely observation-based nature, its inability to adapt to demand variability, and the burden of manually tuning numerous threshold parameters (Wanigasooriya and Ekanayake, 2026). A KEDA-based implementation driven by application-level Prometheus metrics partially addresses all three concerns: it provides an application-aware trigger rather than an infrastructure-level observation, it can be parameterised against workload-specific signals that vary with the demand pattern of the Redis Cluster, and it reduces manual threshold tuning by anchoring scaling decisions to semantically meaningful metrics.

3.4 Scope and Expected Outcome.

Given this theoretical and empirical foundation, this thesis hypothesises that the KEDA-driven operator approach will exhibit measurably lower response latency under load spikes, fewer instances of cluster-level resource saturation, and a more stable Redis Cluster topology compared to the naive HPA baseline. The experimental evaluation presented in Chapter 7 is designed to test this hypothesis directly, using Grafana k6 to generate controlled, reproducible traffic patterns against both configurations and Prometheus-backed Grafana dashboards to measure the resulting autoscaling behaviour and service performance.

4 Technologies

This section provides a brief overview of each technology that constitutes the experimental infrastructure used in this thesis. Together these components form a reproducible, cloud-native testbed for evaluating the two autoscaling strategies under investigation.

4.1 K3s

K3s is a CNCF-certified, lightweight Kubernetes distribution developed by Rancher. It packages the full Kubernetes control plane into a single binary of less than 70 MB and eliminates several heavyweight dependencies of upstream Kubernetes, making it well-suited for resource-constrained environments such as edge devices, IoT deployments, and developer workstations (K3s Project, 2024). Despite its reduced footprint, K3s is fully conformant with the Kubernetes API and supports the complete set of Kubernetes primitives required by this thesis, including *StatefulSets*, *CustomResourceDefinitions*, and Helm chart deployments. K3s serves as the cluster management layer for all infrastructure described in this thesis.

4.2 FastAPI

FastAPI is a modern, high-performance Python web framework designed for building RESTful APIs. It is built on top of *Starlette* for its web-server layer and *Pydantic* for data validation, and it uses Python type hints to enable automatic request validation and interactive OpenAPI documentation generation (FastAPI, 2024b). FastAPI is built around the Asynchronous Server Gateway Interface (ASGI), enabling non-blocking I/O operations that are especially advantageous in workloads involving concurrent database or cache interactions (FastAPI, 2024a). In the context of this thesis, a FastAPI application serves as the Hypertext Transfer Protocol (HTTP) entrypoint to the cluster, receiving incoming requests from the external load generator and routing read and write operations to the Redis Cluster backend.

4.3 Redis Cluster

Redis is an open-source, in-memory data structure store widely used as a cache, session store, and real-time data platform (Redis Ltd., 2024a). The Redis Cluster topology used in this thesis distributes data across multiple primary nodes using a hash-slot-based sharding mechanism. The full keyspace is divided into $16\,384$ hash slots, each assigned to a primary node, and each primary is optionally backed by one or more replica nodes for fault tolerance (Redis Ltd., 2024c). Redis Cluster provides high availability through automatic failover: if a primary becomes unreachable, one of its replicas is promoted. In this thesis, the Redis Cluster is deployed as a Kubernetes *StatefulSet* with a minimum of three primary nodes and forms the core stateful workload under performance evaluation.

4.4 Horizontal Pod Autoscaler (HPA)

The Horizontal Pod Autoscaler is a built-in Kubernetes controller that automatically adjusts the number of pod replicas in a workload based on observed resource metrics, most commonly CPU and memory utilisation (Kubernetes Documentation, 2024a). The HPA periodically queries the Kubernetes Metrics Server (or a custom metrics adapter) and computes a desired replica count proportional to the ratio of current to target utilisation. Its synchronisation loop runs every 15 seconds by default. While effective for many stateless use cases, the HPA is a fundamentally *reactive* mechanism: scaling decisions are always downstream of observed resource conditions, making it vulnerable to the lag issues described in Section 2.2. In this thesis, the HPA is deployed as the *naive* baseline scaling strategy, configured to scale the Redis *StatefulSet* in response to CPU and memory utilisation thresholds.

4.5 KEDA

Kubernetes Event-Driven Autoscaling (KEDA) is a CNCF-graduated open-source project that extends the Kubernetes HPA with the ability to scale workloads based on event sources and external metrics beyond simple CPU and memory (KEDA Project, 2024a). KEDA introduces the *ScaledObject* custom resource, which declares a scaling target (such as a *Deployment* or *StatefulSet*), a polling interval, cooldown periods, and one or more *scalers* that define the metric source (KEDA Project, 2024c). When the configured metric from the scaler exceeds a threshold, KEDA adjusts the replica count of the target workload by updating the underlying HPA on the user's behalf. In this thesis, KEDA is configured with a Prometheus scaler that reads application-level metrics — such as request queue depth and cache hit ratio — directly from the Prometheus monitoring stack, enabling proactive, context-aware scaling decisions before resource exhaustion occurs (KEDA Project, 2024b).

4.6 Prometheus and Grafana

The observability stack in this thesis is deployed using the *kube-prometheus-stack* Helm chart, which bundles Prometheus, Grafana, and Alertmanager into a single, opinionated release (Prometheus Community, 2024). *Prometheus* is an open-source monitoring system and time-series database that collects metrics by periodically scraping HTTP endpoints exposed by instrumented targets (Prometheus Authors, 2024). In the Kubernetes context, it discovers scrape targets automatically via Kubernetes service discovery, collecting cluster-level metrics from *kube-state-metrics* and *node-exporter*, as well as application-level metrics from the FastAPI and Redis exporters. *Grafana* is an open-source analytics and visualisation platform that queries Prometheus via its native data source integration and renders the collected time-series data as configurable dashboards (Grafana Labs, 2024b). Grafana dashboards are used throughout the experimental evaluation in this thesis to visualise autoscaling behaviour, latency distributions, and resource utilisation over time. Crucially, the Prometheus instance also acts as the external metric backend for the KEDA Prometheus scaler, creating a

unified data path from application telemetry to autoscaling decisions.

4.7 Helm

Helm is the de-facto package manager for Kubernetes. It introduces the concept of a *chart* — a collection of templated Kubernetes manifests that describe a deployable application — and a *release*, which is a running instance of a chart in a cluster (Helm Project, 2024). Helm charts enable repeatable, version-controlled deployments with configurable values overrides and built-in rollback support, significantly reducing the operational complexity of deploying multi-resource applications such as the *kube-prometheus-stack*. In this thesis, Helm is used to deploy the observability stack, and the deployment configurations are parameterised through values files that are committed alongside the thesis infrastructure code.

4.8 Grafana k6

Grafana k6 is an open-source, developer-focused performance and load testing tool (Grafana Labs, 2024c). Tests in k6 are written as JavaScript modules, and k6 executes them using a Go runtime which maps each *virtual user* (VU) to a lightweight goroutine, enabling high concurrency with low resource overhead (Grafana Labs, 2024d). k6 supports configurable test scenarios including ramp-up and ramp-down of virtual users over time, making it possible to simulate realistic traffic patterns such as gradual load increases, sustained peaks, and sudden spikes (Grafana Labs, 2024a). In the context of this thesis, k6 runs *outside* the Kubernetes cluster as an independent load generator. It sends configurable volumes of HTTP traffic through the FastAPI entrypoint and into the Redis Cluster, producing the synthetic workload against which both autoscaling strategies — HPA and KEDA — are evaluated. The ability to precisely control the number of virtual users and the ramp profile makes k6 well-suited for generating reproducible, parameterised load scenarios that can isolate the performance characteristics of each scaling approach.

5 Test Methodology

To rigorously evaluate the architectural limitations of native Kubernetes scaling primitives against an Operator-driven approach, this thesis adopts a deterministic, A/B comparative methodology. By intentionally eschewing active traffic generators and dynamic metric-driven autoscalers, the experimental design isolates the fundamental data safety characteristics of stateful scale-in operations. This methodology distils the evaluation to the overarching contrast between Kubernetes' reverse-ordinal pod deletion and the topology-aware data migration orchestrated by a domain-specific operator (The Kubernetes Authors, n.d.-a; OpsTree Solutions, n.d.).

5.1 Definition of Success

A foundational aspect of this methodology is the establishment of a rigorous, state-aware metric for what constitutes a successful scaling event. In a stateful context, capacity reduction without data integration is functionally equivalent to system failure. Therefore, a successful scale-in is strictly defined by three absolute criteria: maintaining a healthy `cluster_state:ok` status, ensuring that all $16\,384$ hash slots remain actively covered by the surviving nodes, and demonstrating zero data loss as evidenced by the perfect retention of the dataset and the total absence of `CLUSTERDOWN` errors during post-transition read probes (Redis Ltd., 2024c).

5.2 A/B Comparative Structure

The methodology relies on an A/B comparative analysis designed to eliminate confounding variables by enforcing identical physical infrastructure and initial cluster geometries (The Kubernetes Authors, n.d.-b). Both scenarios initialize a strictly configured four-node Redis cluster composed exclusively of master nodes, deliberately omitting replica nodes to ensure that any topological disruption immediately surfaces as measurable data loss rather than being temporarily absorbed by high-availability failover mechanisms.

- **Scenario A (Native Primitives):** Simulates the mechanical action of a Horizontal Pod Autoscaler by directly decrementing the replica count of the underlying `StatefulSet`. This scenario tests the hypothesis that Kubernetes' native, topology-blind termination sequence inherently violates the data integrity requirements of a sharded database.
- **Scenario B (Operator Pattern):** Modifies the declarative `clusterSize` parameter of the `RedisCluster` Custom Resource. This scenario tests the hypothesis that encoding domain-specific orchestration knowledge into a Kubernetes controller enables the pre-emptive, topology-aware resharding necessary for a graceful cluster downscale.

5.3 High Reproducibility and Empirical Objectivity

Scientific reproducibility is enforced through strict execution hygiene. Every experimental iteration is preceded by a programmatic "scorched earth" initialization phase that systematically uninstalls all Helm releases, purges the designated namespaces, and deletes all persistent volume claims. This absolute state destruction guarantees that no residual configurations or orphaned data fragments from prior executions can contaminate the baseline measurements.

Furthermore, by performing the scaling operations under strictly quiescent conditions—where a precisely enumerated dataset of 5000 keys is populated prior to scaling, and no concurrent external write load is applied—the methodology achieves absolute empirical objectivity. It prevents intermittent network contention, connection tracking saturation, and load-induced resharding interruptions from confounding the results, thereby yielding a pristine measurement of the underlying scaling mechanisms' inherent capabilities.

6 Experiment Architecture and Test Flow

This chapter details the architecture of the experimental testbed and the procedural flow employed to evaluate the data safety characteristics of stateful scaling mechanisms. To rigorously isolate the fundamental mechanical differences between native Kubernetes scaling and topology-aware operator-driven scaling, the experimental design eschews dynamic, load-driven autoscaling triggers in favor of a quiescent, deterministic proof of concept. By explicitly removing external load generators, application entry-points, and automated metric evaluations, the experiment eliminates confounding variables—such as network saturation, connection tracking limits, or resharding interruptions caused by live writes—thereby distilling the evaluation entirely down to the preservation of data integrity and cluster availability during a scale-in event.

6.1 System Architecture

The underlying infrastructure consists of a lightweight Kubernetes distribution, `k3s`, deployed across a physically distributed cluster of six resource-constrained mini-PCs (K3s Project, 2024). These hardware constraints model a realistic edge or on-premises environment where efficient resource reclamation is critical. Unlike dynamic scenarios that require external load injection routing through `NodePort` services, this quiescent architecture operates entirely within the cluster boundaries.

The architecture comprises two mutually exclusive data tier deployments, which represent the primary independent variable of the experiment:

- **Native Primitives (HPA Scenario):** The Redis Cluster is assembled manually using foundational Kubernetes resources. A `StatefulSet` manages the pod lifecycle, utilizing a headless `Service` to provide stable network identities for intra-cluster communication (The Kubernetes Authors, n.d.-b). Because the declarative `StatefulSet` manifest cannot express the topological geometry of a Redis Cluster, the hash slots and master-replica assignments must be imperatively configured via the `redis-cli` binary during initialization (Redis Ltd., 2024c).
- **Operator Pattern (KEDA Scenario):** The data tier is deployed declaratively using the Opstree Redis Operator (OpsTree Solutions, n.d.). The operator's custom controller resides within the cluster and continuously watches a `RedisCluster` Custom Resource, autonomously translating the requested `clusterSize` parameter into underlying `StatefulSet` modifications, `CLUSTER MEET` commands, and coordinated hash slot migrations.

To provide continuous, objective telemetry during the scaling transitions, a unified observability stack is deployed via the `kube-prometheus-stack` Helm chart. Prometheus scrapes the cluster-state metrics directly from the `redis-cli` processes and exporter sidecars, while Grafana visualizes the exact sequence of topological changes, slot allocations, and data retention metrics.

6.2 Configuration Manifests

To instantiate the environments described above, the experiment relies on declarative YAML manifests. Listing 1 details the baseline native Kubernetes configuration, explicitly demonstrating the lack of topological awareness in a standard `StatefulSet`. Listing 2 displays the Helm values utilized to configure the Opstree Redis Operator, showcasing the abstraction of cluster sizing and exporter injection.

Listing 1: Kubernetes manifests for the native HPA Redis scenario (`redis-hpa-cluster.yaml`).

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: redis-min-config
  namespace: redis
data:
  redis.conf: |
    cluster-enabled yes
    cluster-config-file nodes.conf
    cluster-node-timeout 5000
    appendonly no
---
apiVersion: v1
kind: Service
metadata:
  name: redis-headless
  namespace: redis
  labels:
    app: redis
spec:
  clusterIP: None
  ports:
    - port: 6379
      name: redis
    - port: 16379
      name: gossip
    - port: 9121
      name: metrics
  selector:
    app: redis
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: redis
  namespace: redis
spec:
  serviceName: redis-headless
  # 6 pods = 3 masters + 3 replicas. The master/replica ROLES
    ↪ are assigned at runtime by the
  # setup script's 'redis-cli --cluster create
    ↪ --cluster-replicas 1' - a vanilla StatefulSet
  # cannot express "pod X replicates pod Y", which is exactly
    ↪ the topology gap the operator
  # closes declaratively. The HPA's minReplicas is pinned to 6
    ↪ (see redis-hpa-scaling.yaml) so
  # it won't scale the StatefulSet down to 3 while idle and
    ↪ strip the replicas before the test.
  replicas: 6
```

```
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
        - name: redis
          image: redis:7.2-alpine
          command: ["redis-server", "/etc/redis/redis.conf"]
          resources:
            ## Base requests are strictly required for HPA to
               ↪ calculate usage percentages
            requests:
              cpu: "100m"
              memory: "128Mi"
          volumeMounts:
            - name: conf
              mountPath: /etc/redis
        - name: redis-exporter
          image: ghcr.io/oliver006/redis_exporter:latest
          ports:
          - containerPort: 9121
            name: metrics
          env:
          # Since it's a sidecar, it can reach the main Redis
             ↪ container via localhost
          - name: REDIS_ADDR
            value: "redis://localhost:6379"
          resources:
            requests:
              cpu: "50m"
              memory: "64Mi"
      volumes:
        - name: conf
          configMap:
            name: redis-min-config
```

Listing 2: Helm values configuring the Opstree Redis Operator (`redis-operator-cluster-values.yaml`).

```
redisCluster:
  clusterSize: 3
  # IMPORTANT: do NOT override this with the vanilla redis
     ↪ image. The Opstree operator
  # depends on its OWN image's entrypoint
     ↪ (quay.io/opstree/redis) to enable cluster mode
  # (cluster-enabled yes). A plain redis:7.2-alpine boots in
     ↪ STANDALONE mode, so redis-py
  # fails with "Cluster mode is not enabled on this node" and
     ↪ the entrypoint never connects.
  # We therefore use the chart's default Opstree image; the
     ↪ minor version delta vs the HPA
  # scenario's redis:7.2 is an acceptable, documented trade-off
     ↪ for a working cluster.
  resources:
    requests:
      cpu: "100m"
      memory: "128Mi"
```

```
    limits:
      cpu: "250m"
      memory: "256Mi"
  leader:
    # Un-pin leader replicas so clusterSize (driven by KEDA)
      ↪ controls the master count.
    replicas: null
    affinity:
      podAntiAffinity:
        # Preferred (not required) so a scale-out to 4-6 masters
          ↪ can't get stuck Pending
        # if a node is momentarily unschedulable - the scheduler
          ↪ still spreads leaders
        # one-per-node when it can. weight 100 = strongest
          ↪ preference.
        preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          podAffinityTerm:
            labelSelector:
              matchLabels:
                app: redis-cluster-leader
            topologyKey: kubernetes.io/hostname

  # Same un-pin for followers, so clusterSize drives the whole
    ↪ cluster symmetrically.
  follower:
    replicas: null

# Exporter sidecar: source of redis_keyspace_hits_total /
  ↪ _misses_total that KEDA's
# cache-miss-ratio trigger reads. Mirrors the oliver006 exporter
  ↪ used in the HPA
# scenario for identical metric semantics.
redisExporter:
  enabled: true
  image: ghcr.io/oliver006/redis_exporter
  tag: latest
  resources:
    requests:
      cpu: "50m"
      memory: "64Mi"

serviceMonitor:
  enabled: true
  interval: 15s
  scrapeTimeout: 10s
  extraLabels:
    release: prometheus
```

6.3 Experimental Test Flow

The empirical evaluation is driven by an automated, deterministic execution pipeline (`data-safety-experiment.sh`) designed to enforce identical baseline conditions across both experimental scenarios. The pipeline advances through five strict, verifiable phases:

Phase 1: Deterministic Environment Initialization. To guarantee that residual state from prior executions does not contaminate the findings, every test iteration begins with a comprehensive environment purge. The pipeline systematically uninstalls all Helm releases, deletes the relevant Kubernetes namespaces, and destroys all `PersistentVolumeClaims` associated with the Redis workloads. The execution halts until Kubernetes confirms the absolute termination of all prior resources, ensuring a sterile baseline.

Phase 2: Baseline Provisioning and Geometry Validation. The designated data tier is deployed and configured to form a four-node cluster consisting exclusively of master nodes, deliberately omitting replicas to ensure that any topological failure directly manifests as a measurable data cliff rather than being temporarily masked by high-availability failover mechanisms. In the native primitive scenario, the `StatefulSet` is scaled to four, and the `redis-cli -cluster create` command is executed to allocate $4{,}096$ hash slots to each node. In the operator scenario, the `RedisCluster` resource is applied with a `clusterSize` of four, delegating the cluster formation to the custom controller. The pipeline subsequently polls the primary node until the cluster explicitly reports a `cluster_state:ok` status with all $16{,}384$ hash slots successfully covered.

Phase 3: Quiescent Data Population. Following the successful validation of the cluster geometry, the pipeline injects a highly deterministic, verifiable dataset. Utilizing the `redis-cli` binary acting as the sole writer within the cluster, exactly 5000 distinct keys are populated sequentially. Because the workload utilizes standard key distributions without hash tags, the dataset is uniformly striped across all four master nodes. The absence of any concurrent read or write traffic guarantees that the cluster enters the subsequent scaling phase in a perfectly quiescent, stable state.

Phase 4: Controlled Scale-In Execution. To isolate the execution mechanism from the latency and heuristic variations of an active autoscaler, the scaling trigger is invoked imperatively. This phase directly simulates the exact system calls that the HPA or KEDA controllers would generate when commanding a downscale from four nodes to three:

- **In the native scenario:** The `StatefulSet` replica count is patched from four to three, triggering Kubernetes to terminate the highest-ordinal pod (`redis-3`) in strict accordance with reverse-ordinal deletion rules (The Kubernetes Authors, n.d.-b).
- **In the operator scenario:** The `clusterSize` parameter within the `RedisCluster` resource is patched from four to three, prompting the operator's reconciliation loop to orchestrate a data drain prior to pod termination.

Phase 5: Integrity Verification and Telemetry Capture. Immediately following the successful termination of the target pod, the pipeline issues a cluster state verification

request to the surviving primary node. The experiment programmatically records the updated `cluster_state`, the total number of successfully assigned hash slots, and the surviving key count. To actively verify availability, the pipeline issues read probes against specific keys distributed throughout the keyspace, monitoring for `CLUSTERDOWN` errors that indicate a compromised topological state resulting from orphaned slots.

7 Results and Analysis

This chapter presents the empirical findings of the data-safety experiment. To isolate the mechanical differences between native Kubernetes scaling and operator-driven scaling, the evaluation was conducted under quiescent conditions (zero external write pressure). A deterministic baseline was established for both scenarios: a Redis Cluster deployed with four primary master nodes and exactly 5000 keys uniformly distributed across the keyspace. The independent variable is the mechanism used to scale the cluster inward from four masters to three.

7.1 Scenario A: Native HPA Scale-In and the Data Cliff

In the native Horizontal Pod Autoscaler (HPA) scenario, the scaling action was triggered by directly reducing the `StatefulSet` replica count from four to three. This mirrors the exact operational instruction an HPA controller issues when resource utilisation falls below its target threshold.

Because Kubernetes `StatefulSets` terminate pods in strictly descending ordinal order (The Kubernetes Authors, 2024), the scheduler immediately issued a termination signal to the highest-ordinal pod (`redis-3`). The native Kubernetes primitives possess no awareness of the Redis hash slot topology; consequently, the pod was terminated without initiating a `CLUSTER MEET` or migrating its assigned hash slots to the surviving nodes.

As recorded in the test output, the instantaneous removal of `redis-3` triggered a catastrophic data cliff:

- **Slot Orphanage:** Cluster hash slot coverage fell immediately from $16\,384$ to $12\,288$, leaving 4096 slots permanently orphaned.
- **Data Loss:** The total key count dropped from 5000 to 3750. Exactly 1250 keys (25% of the dataset, correlating directly to the loss of one out of four evenly balanced nodes) were permanently deleted.
- **Availability:** The read probes executed immediately following the pod deletion resulted in a 100% failure rate. Because the cluster fell below the full $16\,384$ slot coverage requirement, Redis transitioned into a `CLUSTERDOWN` state, refusing all subsequent read and write traffic (Redis Ltd., 2024d).

These results validate the hypothesis that native Kubernetes scaling primitives provision and terminate compute capacity but are structurally incapable of integrating or safely decommissioning stateful topology.

7.2 Scenario B: Operator-Driven Graceful Scale-In

In the operator scenario, the scale-in was executed by patching the `clusterSize` parameter of the `RedisCluster` Custom Resource from four down to three. This

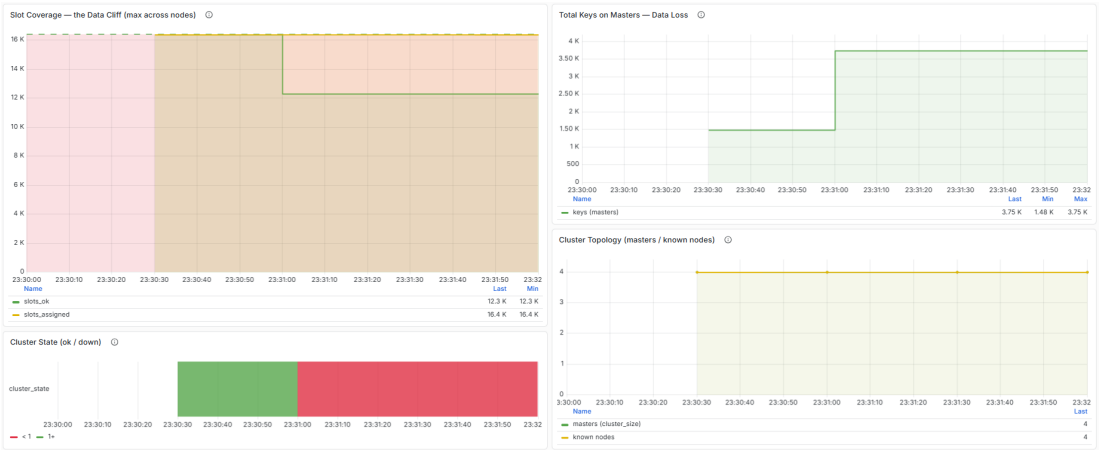

Figure 1: Timeline of the HPA scale-in event. The vertical red line indicates the moment the `StatefulSet` replica count was reduced from four to three, triggering the deletion of `redis-3`. The subsequent drop in total keys and hash slot coverage is visible, along with the spike in read errors due to cluster unavailability.

mirrors the exact action taken by a KEDA `ScaledObject` triggering a downscale event.

Rather than immediately terminating a pod, the Opstree Redis Operator intercepted the state change. The operator's reconciliation loop identified the required topological shift and initiated a live resharding process, draining the 4096 hash slots from the target node and redistributing them evenly among the three surviving masters (Opstree Solutions, 2023). Only after the cluster achieved consensus that the departing node held zero slots did the operator delete the underlying `StatefulSet` pod.

The test output confirms a fully graceful scale-in:

- **Slot Integrity:** Hash slot coverage remained at $16\,384$ throughout and after the scaling event.
- **Data Preservation:** The cluster retained all 5000 keys with zero data loss.
- **Availability:** The read probes executed following the scale-in recorded zero `CLUSTERDOWN` errors, confirming that cluster availability was maintained continuously.

7.3 Synthesis of Findings

The stark contrast between the two scenarios empirically proves the necessity of the operator pattern for stateful workloads. The native HPA pathway demonstrates that allowing a generic infrastructure controller to manage database lifecycle operations results in guaranteed data loss and cluster unavailability.

While the operator successfully bridged the gap between compute provisioning and data topology, it is important to note that this success was recorded under quiescent conditions. As established in the methodology, the operator's live resharding logic

Figure 2: Timeline of the operator-driven scale-in event. The vertical blue line indicates the moment the `clusterSize` parameter was patched from four to three. The graph shows that total keys and hash slot coverage remained stable throughout the resharding process, and no read errors were recorded, confirming continuous availability.

remains vulnerable to interruption if subjected to heavy concurrent write pressure (Redis Ltd., 2024b). Nevertheless, when evaluating pure architectural capability, the operator pattern is the only evaluated mechanism that possesses the domain knowledge required to safely perform a stateful scale-in.

8 Conclusion

Anhang

Hier steht der Anhang.

Glossar

Hier steht das Glossar.

Literaturverzeichnis

Amazon Web Services. (2022). *Container build lens — AWS well-architected framework: Design principles* [Accessed: 2026-01-24]. https://docs.aws.amazon.com/pdfs/wellarchitected/latest/container-build-lens/container-build-lens.pdf#design-principles

Cloud Native Computing Foundation. (2023). *CNCF announces graduation of KEDA* [Accessed: 2026-01-24]. https://www.cncf.io/announcements/2023/08/22/cloud-native-computing-foundation-announces-graduation-of-kubernetes-autoscaler-keda/

Cloud Native Computing Foundation. (2024). *Managing large-scale Redis clusters on Kubernetes with an operator* [Accessed: 2026-01-24]. https://www.cncf.io/blog/2024/12/17/managing-large-scale-redis-clusters-on-kubernetes-with-an-operator-kuaishous-approach/

CNCF TAG App Delivery. (2023). *CNCF operator white paper* [Accessed: 2026-01-24]. https://tag-app-delivery.cncf.io/whitepapers/operator/

Docker Inc. (2016). *Containers are not VMs* [Accessed: 2026-01-24]. https://www.docker.com/blog/containers-are-not-vms/

FastAPI. (2024a). *Concurrency and async / await — FastAPI* [Accessed: 2026-01-24]. https://fastapi.tiangolo.com/async/

FastAPI. (2024b). *FastAPI — modern, fast, web framework for building APIs with python* [Accessed: 2026-01-24]. https://fastapi.tiangolo.com/

Google Cloud. (2023). *Zero-downtime scaling in Memorystore for Redis cluster* [Accessed: 2026-01-24]. https://cloud.google.com/blog/products/databases/zero-downtime-scaling-in-memorystore-for-redis-cluster

Grafana Labs. (2024a). *API load testing — Grafana k6 documentation* [Accessed: 2026-01-24]. https://grafana.com/docs/k6/latest/testing-guides/api-load-testing/

Grafana Labs. (2024b). *Grafana — the open and composable observability platform* [Accessed: 2026-01-24]. https://github.com/grafana/grafana

Grafana Labs. (2024c). *Grafana k6 documentation* [Accessed: 2026-01-24]. https://grafana.com/docs/k6/latest/

Grafana Labs. (2024d). *K6 — a modern load testing tool* [Accessed: 2026-01-24]. https://github.com/grafana/k6

Helm Project. (2024). *Using Helm* [Accessed: 2026-01-24]. https://helm.sh/docs/intro/using_helm/

IBM. (2024). *The history of kubernetes* [Accessed: 2026-01-24]. https://www.ibm.com/think/topics/kubernetes-history

K3s Project. (2024). *K3s — lightweight kubernetes* [Accessed: 2026-01-24]. https://docs.k3s.io/

KEDA Project. (2024a). *KEDA — kubernetes event-driven autoscaling* [Accessed: 2026-01-24]. https://keda.sh/

KEDA Project. (2024b). *KEDA — prometheus scaler* [Accessed: 2026-01-24]. https://keda.sh/docs/2.19/scalers/prometheus/

KEDA Project. (2024c). *KEDA — ScaledObject specification* [Accessed: 2026-01-24]. https://keda.sh/docs/2.19/reference/scaledobject-spec/

KEDA Project. (2024d). *KEDA — scaling deployments, StatefulSets & custom resources* [Accessed: 2026-01-24]. https://keda.sh/docs/2.19/concepts/scaling-deployments/

Kubernetes Documentation. (2024a). *Autoscaling workloads* [Accessed: 2026-01-24]. https://kubernetes.io/docs/concepts/workloads/autoscaling/

Kubernetes Documentation. (2024b). *Disruptions — Pod disruption budgets* [Accessed: 2026-01-24]. https://kubernetes.io/docs/concepts/workloads/pods/disruptions/

Kubernetes Documentation. (2024c). *Kubernetes overview* [Accessed: 2026-01-24]. https://kubernetes.io/docs/concepts/overview/

Kubernetes Documentation. (2024d). *Operator pattern* [Accessed: 2026-01-24]. https://kubernetes.io/docs/concepts/extend-kubernetes/operator/

Kubernetes Documentation. (2024e). *Statefulsets* [Accessed: 2026-01-24]. https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/

Medium / emreblblvv. (2024). *Auto-scaling with KEDA using custom RED metrics from Prometheus* [Accessed: 2026-01-24]. https://medium.com/@emreblblvv/auto-scaling-with-keda-using-custom-red-metrics-from-prometheus-76d50785e442

Mell, P., & Grance, T. (2011). *The NIST definition of cloud computing* (tech. rep. No. Special Publication 800-145) (Accessed: 2026-01-24). National Institute of Standards and Technology. https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-145.pdf

OpsTree Solutions. (n.d.). Redis-operator: A Golang based Redis operator for Kubernetes [Accessed: 2026-06-06].

Opstree Solutions. (2023). Redis operator for kubernetes. https://ot-container-kit.github.io/redis-operator/

Portworx. (2024). *Understanding StatefulSets in Kubernetes* [Accessed: 2026-01-24]. https://portworx.com/knowledge-hub/understanding-statefulsets-in-kubernetes/

Prometheus Authors. (2024). *Prometheus overview* [Accessed: 2026-01-24]. https://prometheus.io/docs/introduction/overview/

Prometheus Community. (2024). *Kube-prometheus-stack Helm chart* [Accessed: 2026-01-24]. https://artifacthub.io/packages/helm/prometheus-community/kube-prometheus-stack

Red Hat. (2025). *Stateful vs. stateless applications* [Accessed: 2026-01-24]. https://www.redhat.com/en/topics/cloud-native-apps/stateful-vs-stateless

Redis Ltd. (2024a). *Redis — real-time data for agents & apps* [Accessed: 2026-01-24]. https://redis.io/

Redis Ltd. (2024b). Redis cluster administration. https://redis.io/docs/management/scaling/

Redis Ltd. (2024c). *Redis cluster specification* [Accessed: 2026-01-24]. https://redis.io/docs/latest/operate/oss_and_stack/reference/cluster-spec/

Redis Ltd. (2024d). Redis cluster specification. https://redis.io/docs/reference/cluster-spec/

The Kubernetes Authors. (n.d.-a). *Operator pattern* [Accessed: 2026-06-06]. https://kubernetes.io/docs/concepts/extend-kubernetes/operator/

The Kubernetes Authors. (n.d.-b). *Statefulsets* [Accessed: 2026-06-06]. https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/

The Kubernetes Authors. (2024). Statefulsets. https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/

Thoras. (2024). *Predictive horizontal pod autoscaling* [Accessed: 2026-01-24]. https://docs.thoras.ai/guides/hpa

Vazquez, A. (2026). *Kubernetes HPA best practices: When CPU works, why memory doesn't* [Accessed: 2026-01-24]. https://alexandre-vazquez.com/kubernetes-hpa-best-practices/

Wanigasooriya, C., & Ekanayake, I. (2026). NimbusGuard: A novel framework for proactive Kubernetes autoscaling using deep Q-networks [Accessed: 2026-01-24]. *arXiv preprint arXiv:2604.11017*. https://arxiv.org/html/2604.11017v1

Stichwortverzeichnis

Hier steht das Stichwortverzeichnis.

Anlagen

Hier sind die Anlagen.